

Chapter 5

Resource Complexity

Often, there will be competing solutions to a given problem. Poor reasons to choose one solution over another include having thought of one first or believing that one will be easier to encode in your preferred programming language. Running programs has a real-world cost, and energy requirements are a major bottleneck in large-scale computation. Keep in mind that a good program is written once, but run thousands or millions of times, so developer time is worth nothing compared to the actual runtime cost of a system. We cannot directly measure the energy cost of an algorithm, especially as they will vary based on runtime environments, but we can, as a proxy for this, analyze how runtime and memory requirements scale relative to the size of the input data. This chapter provides a foundation for doing this kind of analysis, working through some of the procedures introduced earlier in the book, after first introducing a powerful reasoning technique: the principle of *induction*. Inductive reasoning can be used to show program correctness, as will be demonstrated in the first section, but can also be used to understand scaling of resource requirements.

5.1 Mathematical Induction

Ordinary mathematical induction begins from the idea of chaining conclusions. If you can show that a property is true at some base case b , and that whenever that property is true for some value $k \geq b$ you can show it is also true at $k + 1$, you can conclude that it is true at b (because you directly showed that directly), at $b + 1$ (because it was true at b), at $b + 2$ (because it was true at $b + 1$), and so on. You can reach any natural number $n \geq b$ this way, so you conclude that for all $n \geq b$, the property holds. For example, forget for a moment that Java integers are bounded in size and consider our squaring function that embodies the unary representation of numbers, reprinted here. We can use the principle of ordinary mathematical induction to show that this does, indeed, compute n^2 for any nonnegative input n .

52 *<int-to-long squaring 52>*≡

```
public static long squareUnary(int n) {
    if (n == 0) { return 0L; }
    return squareUnary(n - 1) + (long)n + (long)n-1L;
}
```

The structure of the recursion splits cleanly into two cases: the base case and the inductive step. For the base case, we consider $n = 0$: we know that $n^2 = 0$ and this is the value returned by the function. The property holds at the base case. The inductive step begins as follows: assume that for some $k \geq 0$ we know that $\text{squareUnary}(k) = k^2$. We want to show that the property remains true at $k + 1$, that $\text{squareUnary}(k + 1) = (k + 1)^2$. To do so, consider that because $k \geq 0$, we know $k + 1 \neq 0$, so $\text{squareUnary}(k + 1) = \text{squareUnary}(k) + (k + 1) + k$. By our initial assumption, we can simplify this to $k^2 + (k + 1) + k = k^2 + 2k + 1 = (k + 1)^2$, as desired. From this we conclude that for any $n \geq 0$, $\text{squareUnary}(n) = n^2$. Replicated in brief, the argument is as follows.

Base case: at $n = 0$, $\text{squareUnary}(n) = 0 = n^2$. ✓

Inductive step: assume that for some $k \geq 0$, $\text{squareUnary}(k) = k^2$. We want that $\text{squareUnary}(k + 1) = (k + 1)^2$.

$$\begin{aligned} \text{squareUnary}(k + 1) &= \text{squareUnary}(k) + (k + 1) + k \\ &= k^2 + k + 1 + k \\ &= k^2 + 2k + 1 \\ &= (k + 1)^2. \quad \checkmark \end{aligned}$$

Conclusion: for all $n \geq 0$, $\text{squareUnary}(n) = n^2$.

This analysis does not account for the boundedness of integer types in Java. To truly verify correctness, it must further be shown that no undesirable overflow occurs during computation. As the largest `int` value is 2,147,483,647, whose square is 4,611,686,014,132,420,609, which is less than the largest possible `long` (9,223,372,036,854,775,807), we are fine here.

Ordinary mathematical induction does not cleanly map onto all recursive functions, even when inputs are numeric. It is best for numeric functions recurring on the predecessor. Recall that we have also used binary representations, where division by two is natural. *Strong mathematical induction* is equivalent in power, but allows for more assumptions to be made. Instead of assuming that the property is true at just the one value k and showing that it remains true at $k + 1$, you get to assume that it is true for all i where $0 \leq i < k$ and must show that it remains true at k . You might also include multiple base cases. Despite being equivalent to ordinary mathematical induction, this formalism is more useful for analyzing, say, our squaring method based on binary representations, reprinted here.

<int-to-long squaring 53>≡

53

```
public static long squareBinary(int n) {
    if (n == 0) { return 0L; }
    if (n == 1) { return 1L; }
    if (n%2 == 0) { return 4L*squareBinary(n/2); }
    return 4L*squareBinary(n/2) + (long)n + (long)n-1L;
}
```

The structure of this code maps onto a proof by strong mathematical induction.

Base case: at $n = 0$, $\text{squareBinary}(n) = 0 = n^2$. ✓

Base case: at $n = 1$, $\text{squareBinary}(n) = 1 = n^2$. ✓

Strong inductive step: assume that $k > 1$ and for all i where $0 \leq i < k$, $\text{squareBinary}(i) = i^2$.

Case: if k is even, $0 \leq \lfloor \frac{k}{2} \rfloor = \frac{k}{2} < k$.

$$\begin{aligned} \text{squareBinary}(k) &= 4 \cdot \text{squareBinary}\left(\frac{k}{2}\right) \\ &= 4 \cdot \left(\frac{k}{2}\right)^2 \\ &= 4 \cdot \left(\frac{k^2}{4}\right) \\ &= k^2. \checkmark \end{aligned}$$

Case: if k is odd, $0 \leq \lfloor \frac{k}{2} \rfloor = \frac{k-1}{2} < k$.

$$\begin{aligned} \text{squareBinary}(k) &= 4 \cdot \text{squareBinary}\left(\frac{k-1}{2}\right) + k + k - 1 \\ &= 4 \cdot \left(\frac{k-1}{2}\right)^2 + k + k - 1 \\ &= 4 \cdot \left(\frac{(k-1)^2}{4}\right) + k + k - 1 \\ &= (k-1)^2 + k + k - 1 \\ &= k^2 - 2k + 1 + k + k - 1 \\ &= k^2. \checkmark \end{aligned}$$

Conclusion: for all $n \geq 0$, $\text{squareBinary}(n) = n^2$.

5.2 Time Scaling

Instead of using the structure of a recursive function to prove that its value is correct, we can use this structure to reason about how long the function takes to operate, given a particular input. Let $T_{\square}(n)$ represent the time it takes to compute the square of n using `squareUnary`. We can count the individual operations involved. In the base case where n is zero, there is only a comparison against 0 (which succeeds) before the function returns 0. We might say that this is one unit of work: $T_{\square}(0) = 1$. In the recursive case where n is greater than zero, there is this comparison as well as some arithmetic in addition to the recursive call. Perhaps we say that each arithmetic operation is one unit, so for $n > 0$ we have $T_{\square}(n) = T_{\square}(n - 1) + 4$: this accounts for the recursive call plus three arithmetic operations and the comparison.

This *recurrence relation* expresses how many units of work are required to compute the square of n . However, a *closed-form* solution, expressing the result directly without recursion, is more convenient. This is especially true when we want to compare algorithms, as raw operation counts are less meaningful than the rate at which those counts change.

Given a recurrence relation of the form $T(n) = T(n - 1) + E$ with a base case at $T(0) = x$, for some arbitrary expression E , you can expect $T(n) = n \cdot E + x$. So we should expect $T_{\square}(n) = 4n + 1$. It is, however, useful to be able to compute this closed-form solution when the recurrence relation is in other forms. Perhaps the recursive case is not exactly at $n - 1$ or the base case is not at $n = 0$. One way to do so is to compute a few successive values: $T_{\square}(0) = 1$, $T_{\square}(1) = T_{\square}(0) + 4 = 5$, $T_{\square}(2) = T_{\square}(1) + 4 = 9$, and $T_{\square}(3) = T_{\square}(2) + 4 = 13$. From there, look at the differences between successive results: here, in each case, the difference is 4. As this appears to be constant after one iteration, we can assume that the result is linear: for some a and b , we suspect that $T_{\square}(n) = an + b$. Substituting at $n = 0$ we have $1 = a \cdot 0 + b = b$, so $b = 1$. Then substituting at $n = 1$ we have $5 = a \cdot 1 + b = a + 1$, so $a = 4$. If the linearity hypothesis is correct, then $T_{\square}(n) = 4n + 1$. We can use ordinary mathematical induction to prove it.

Base case: at $n = 0$, $T_{\square}(n) = 1 = 4 \cdot 0 + 1$. ✓

Inductive step: assume that for some $k \geq 0$ it holds that $T_{\square}(k) = 4k + 1$.

We want that $T_{\square}(k + 1) = 4(k + 1) + 1$.

$$\begin{aligned} T_{\square}(k + 1) &= T_{\square}(k) + 4 \\ &= 4k + 1 + 4 \\ &= 4k + 4 + 1 \\ &= 4(k + 1) + 1. \quad \checkmark \end{aligned}$$

Conclusion: for all $n \geq 0$, $T_{\square}(n) = 4n + 1$.

We are primarily concerned with growth rates. For a linear function like this, for sufficiently large n , we can say that doubling n approximately doubles the runtime of the function. Here, $T_{\square}(2n) \approx 2T_{\square}(n)$ for large n . For a concrete example, consider that $T_{\square}(50,000) = 200,001$ and $T_{\square}(100,000) = 400,001$; their quotient is $\frac{400,001}{200,001} = 1.999995$, very nearly 2.

The analysis for the squaring method based on binary representations is not terribly different if you consider the size of the *representation* rather than the size of the *value*. The base case (one bit) does at most some constant amount of work, while the recursive case (larger numbers) computes a result for a representation one bit smaller and then does some constant amount of work to build up the full result. The runtime of the function is proportional to the number of bits used to represent n , which (for positive n) is $\lceil \log_2(n + 1) \rceil$.

When talking about performance scaling, it is generally the size of the representation that is under consideration. But looking at this function in terms of its value can give an idea of what a recurrence relation looks like when the runtime scales logarithmically with the input. Let $T_{\boxplus}(n)$ represent the runtime for this function, in terms of the input value n . The recursive case halves the input value for the recursive call, and then does some constant amount of additional work (perhaps three work-units) to build the full solution: $T_{\boxplus}(n) = T_{\boxplus}(\lfloor \frac{n}{2} \rfloor) + 3$. How many recursive calls are required to reach a base case? Here that's the number k of times you need to divide by 2 to reach 1: $n/2^k = 1$ means $n = 2^k$. Take the base-2 logarithm of both sides to see $k = \log_2 n$. That is how many times the additional work is performed, plus there is the base case at $T_{\boxplus}(1) = 1$ contributing some work of its own. In this case, we would say $T_{\boxplus}(n) \approx 3 \log_2 n + 1$. This is an approximation. The smooth logarithm does not exactly capture the stair-stepped rounding of the actual $T_{\boxplus}$ function, where $T_{\boxplus}(16) = T_{\boxplus}(30) = 13$. The approximation is correct at powers of two, like $T_{\boxplus}(16) = 13$ and $T_{\boxplus}(32) = 16$, but between these inputs the true values remain stagnant while the approximation smoothly increases.

In general, when the recursive call is some fraction of n and there is constant additional work added, the runtime scales logarithmically. Doubling the size of the input results in only some constant additional work. The squaring method based on binary representations requires time linear in the size of the representation but time logarithmic in the size of the represented value. When dealing with data structures, the size of the representation (such as the number of elements in a list) is usually a more important consideration than the particular values of those elements. However, in comparing our squaring methods, it does make sense to talk about the values: for a given value n , `squareBinary`(n) is likely to be significantly faster than `squareUnary`(n).

This section has been intentionally handwavy with quantifying amounts of work. We are not so much concerned with raw numbers, but with how solutions scale. The next section formalizes this notion.

Table 5.1: Common growth rates, in increasing order.

name	constant	logarithmic	linear	linearithmic	quadratic	cubic	exponential
$g(n)$	1	$\log n$	n	$n \log n$	n^2	n^3	2^n

5.3 Bachmann–Landau Notation

Sometimes called *big-O notation*, the *Bachmann–Landau notation* describes compactly how a function scales relative to another. The fundamental definition is of the following:

$$f(n) = O(g(n))$$

This is an abuse of notation in many ways, but what it means is that for some fixed integers N and c that you get to choose, whenever $n > N$ it is also true that $|f(n)| < c \cdot g(n)$. As we are usually concerned only with functions whose output is positive (such as the amount of time or space required to execute an operation), dropping the absolute value is usually fine. In short, $f(n) = O(g(n))$ means that when the inputs are sufficiently large, the (absolute) value of f is less than some fixed multiple of the (absolute) value of g . You can read $f(n) = O(g(n))$ as “the order of f is at most that of g ”, where *order* means, roughly, growth rate.

Under this analysis, only the highest-order terms matter. For example, for any linear function $f(n) = a \cdot n + b$ we have $f(n) = O(n)$, because when $n > b$ we have $a \cdot n + b < a \cdot n + n = (a+1) \cdot n$, which is in turn less than $c \cdot n$ for $c = |a| + 2$. Similarly, for any quadratic function $f(n) = x \cdot n^2 + y \cdot n + z$ we have $f(n) = O(n^2)$, because when $n > y$ we have $x \cdot n^2 + y \cdot n < x \cdot n^2 + n \cdot n = (x+1) \cdot n^2$ and when it is also true that $n > z$ we have $(x+1) \cdot n^2 + z < (x+1) \cdot n^2 + n \leq (x+1) \cdot n^2 + n^2 = (x+2) \cdot n^2$. So by choosing $c = |x| + 3$ we have that $|x \cdot n^2 + y \cdot n + z| < c \cdot n^2$. The notation itself intrinsically discards constant multipliers, and, when inputs are sufficiently large, the lower-order terms do not contribute enough to be meaningful.

For much the same reason, if $f(n)$ and $g(n)$ are polynomials such that the degree of f is less than or equal to that of g , it will always be true that $f(n) = O(g(n))$. So $n = O(n^2)$. But it is not the case that $n^2 = O(n)$: for any constant c , we have that $n^2 > c \cdot n$ whenever $n > c$.

Typically the function $g(n)$ is chosen to be as simple as possible. The most common growth rates are summarized in Table 5.1. Growth rates that are further to the left are *asymptotically better* than those that are further to the right. We say that f is asymptotically better than g when $f(n) = O(g(n))$ but $g(n) \neq O(f(n))$. There are two other important symbols in Bachmann–Landau notation: $f(n) = \Omega(g(n))$ means $g(n) = O(f(n))$, and $f(n) = \Theta(g(n))$ means that both $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ simultaneously. If $f(n) = \Omega(g(n))$ but $g(n) \neq \Omega(f(n))$, we say that f is *asymptotically worse* than g , and if $f(n) = \Theta(g(n))$ we say that f is *asymptotically equivalent* to g . For instance, $\log n$ is asymptotically equivalent

to $\log n^2$, as $\log n^2 = 2 \log n$ and constant factors are irrelevant to Bachmann–Landau notation. For the same reason, the base of the logarithm is unimportant; $\log_b n = \frac{1}{\log b} \log n$, no matter which base is used on the right.

In short, asymptotically, O means “no worse than”, Ω means “no better than”, and Θ means “equivalent to”. It is important to note that these comparisons only apply as the inputs grow large. When inputs are smaller than the necessary size threshold, an algorithm that is asymptotically worse than another may, in fact, perform better.

5.4 Lists

Consider the following complete, simplified linked list class. You can get the size, add an element to the beginning, remove the first element, get the first element, or get the element at an arbitrary index.

(lists/LinkedList.java 57)≡

57

```
public class LinkedList<E> {
    (singly linked list internal node 58)

    private Node<E> head;
    public LinkedList() { head = null; }

    public int size() {
        if (head == null) { return 0; }
        return head.size();
    }

    public void prepend(E x) { head = new Node<>(x, head); }

    public E drop() {
        if (head == null) {
            throw new java.util.NoSuchElementException();
        }
        E out = head.getData();
        head = head.getRest();
        return out;
    }

    public E get() { E out = drop(); prepend(out); return out; }

    public E get(int index) {
        if (head == null) { throw new IndexOutOfBoundsException(); }
        return head.get(index);
    }
}
```

```

58 <ingly linked list internal node 58>≡ (57)
    private static class Node<T> {
        private final T data;
        private final Node<T> rest;

        public Node(T x, Node<T> list) { data = x; rest = list; }
        public T getData() { return data; }
        public Node<T> getRest() { return rest; }

        private int size() {
            if (rest == null) { return 1; }
            return 1 + rest.size();
        }

        private T get(int index) {
            if (index == 0) { return getData(); }
            if (rest == null) {
                throw new IndexOutOfBoundsException();
            }
            return rest.get(index - 1);
        }
    }

```

The `prepend`, `drop`, and zero-argument `get` methods are not recursive. Each operates in constant time. The single-argument `get` method makes as many calls as the index specifies, which, in the worst case, is the same as the number of elements in the list. This means that this last method requires linear time. Similarly, the `size` method requires linear time.

5.4.1 Random-Access Lists

A linked list consists of a spine that is a chain of nodes, where each of these nodes reaches out to reference some data. The spine of a singly linked list directly encodes the size of the list in unary. Each node is one tally mark, holding one piece of data. We can also consider a binary representation.

The binary number system is a place-value system, where the position of a bit determines how much it is worth. The least-significant bit is worth one, the next most significant bit is worth two, the next is worth four, and so on. Translating this into lists, a bit may be zero, holding no data, or it may be one, holding as many data points as its position says it should be worth. We begin with the least significant bit, a node holding either zero values or one value, and double the capacity for each subsequent node. A node is either completely full or entirely empty, never partially filled. As a node need not necessarily hold data, we can maintain an invariant that there always be at least one node; this simplifies the list class itself as the `head` is never null. We also guarantee that the last node for every nonempty list holds data.

<lists/RandomAccessList.java 59a>≡

59a

```
public class RandomAccessList<E> {
    <random-access list internal node 59c>
    <random-access list pair 59b>
    private Node<E> head;
    public RandomAccessList() { head = new Node<>(null); }

    public int size() { return head.size(); }
    public void prepend(E x) { head = head.prepend(x); }
    public E drop() {
        Pair<Node<E>, E> result = head.drop();
        head = result.first;
        if (head == null) { head = new Node<>(null); }
        return result.second;
    }
    public E get() { return get(0); }
    public E get(int index) { return head.get(index); }
}
```

There are two private internal helper classes. Like a regular, singly-linked list, there is a `Node` that holds data, although it is structured differently. There is also a `Pair` class, which is used to allow for doubling the capacity of each later node.

<random-access list pair 59b>≡

(59a 63b) 59b

```
private static class Pair<A,B> {
    public final A first;
    public final B second;
    public Pair(A x, B y) { first = x; second = y; }
}
```

<random-access list internal node 59c>≡

(59a) 59c

```
private static class Node<T> {
    private boolean hasData;
    private T data;
    private Node<Pair<T,T> > rest;

    public Node() { this(null); }
    public Node(Node<Pair<T,T> > list) {
        hasData = false;
        data = null;
        rest = list;
    }
    public Node(T x, Node<Pair<T,T> > list) {
        hasData = true;
        data = x;
        rest = list;
    }
}
<random-access list node methods 60a>
}
```

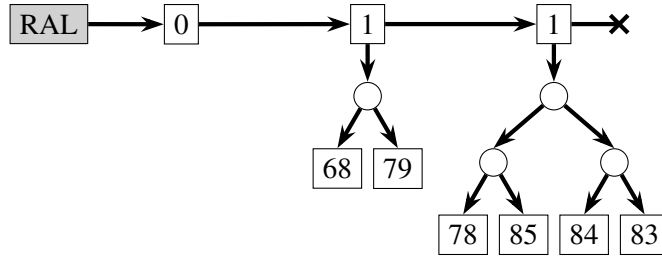


Figure 5.1: A six-element random-access list.

Figure 5.1 depicts a *random-access list* with six nodes. Note that 6 in base ten, written 6_{10} to avoid ambiguity, is 110 in binary, written 110_2 . The nodes are linked from least- to most-significant, and are represented by their value. Nodes with a 1 bit are linked to their content: for an initial node this would be a direct link to data, but for subsequent nodes it is a pair, or a pair of pairs, and so on. Pairs are represented by circles here, and the real data values are in boxes.

To compute the size of this list is to read the spine as a binary number. This takes as much time as there are bits in the number of stored elements, which we have already said is logarithmic in that value. The size method takes time logarithmic in the size of the list. In general, methods on random-access list node will have four cases: whether there are remaining bits might matter, and whether the current bit is zero or one might matter, so we account for all four combinations.

```

60a <random-access list node methods 60a>≡ (59c) 60b>
    private int size() {
        if (rest == null) {
            if (hasData) { return 1; }
            return 0;
        }
        if (hasData) { return 1 + 2*rest.size(); }
        return 2*rest.size();
    }

```

To prepend an element requires incrementing a binary number. If the least-significant bit in the number being considered is zero, it becomes one and takes on the new data. Otherwise, the bit is one and contains data: it becomes zero and we “carry” the pair built from the new data and this original data into the rest of the list.

```

60b <random-access list node methods 60a>+≡ (59c) <60a 61a>
    private Node<T> prepend(T x) {
        if (!hasData) { return new Node<>(x, rest); }
        Pair<T,T> carry = new Pair<>(x, data);
        if (rest == null) { return new Node<>(new Node<>(carry, null)); }
        return new Node<>(rest.prepend(carry));
    }

```

To remove the first element is to reverse the process of prepending an element. If the least-significant bit is one, we want to return the contained data. If it is zero, we need to borrow data from the next bit: it will be a pair; the second element is the data to keep, while the first is the data to remove and return. As we are working with persistent nodes, it is important to return any newly constructed nodes, so the method returns a pair.

```

<random-access list node methods 60a>+≡ (59c) <60b 61b> 61a
private Pair<Node<T>, T> drop() {
    if (hasData) {
        if (rest == null) { return new Pair<>(null, data); }
        return new Pair<>(new Node<>(rest), data);
    }
    Pair<Node<Pair<T,T> >, Pair<T,T> > borrow = rest.drop();
    Node<Pair<T,T> > list = borrow.first;
    T out = borrow.second.first;
    T kept = borrow.second.second;
    return new Pair<>(new Node<>(kept, list), out);
}

```

Finally, to get an arbitrary element, we consider the desired index. If the desired index is zero and we are at an element that contains data, return that data. Otherwise, consider that the rest of the list is a list of paired values. We want to grab the appropriate pair. So subtract one from the index if the current node contained data (to indicate we have passed that one); this is the remaining index. Halve that, rounding down, then get the pair from this resulting position in the rest of the list. If the remaining index were even, we want the first element of that pair, else we want the second.

```

<random-access list node methods 60a>+≡ (59c) <61a 61b
private T get(int index) {
    int remaining = index;
    if (hasData && index == 0) { return data; }
    if (hasData) { remaining = index - 1; }
    if (rest == null) { throw new IndexOutOfBoundsException(); }
    Pair<T,T> p = rest.get(remaining / 2);
    if (remaining%2 == 0) { return p.first; }
    return p.second;
}

```

Each of the `size`, `prepend`, `drop`, and `get` methods here requires, in the worst case, time proportional to the number of nodes along the spine of the list, the number of bits in the list's size. That is, each of these runs in time logarithmic in the size of the list. For `size`, this is always the case, because it reads out this spine as a binary number. For `prepend`, this happens when every bit is 1, that is, when the size of the list is one less than a power of two. For `drop` or for `get`, this is sure

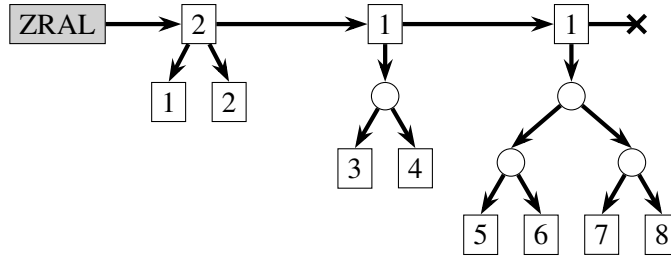


Figure 5.2: An eight-element zeroless random-access list.

to happen when only the most-significant bit is 1, that is, when the size of the list is exactly a power of two.

5.4.2 Zeroless Random-Access Lists

Two issues with the standard binary number system are apparent. First, to get the initial element is no longer a constant-time operation, as we may need to pass through several zero bits before finding any data at all. Second, numbers do not have a unique representation; leading zeros do not bear data, do not change the content of the list, but can be represented. This is why the *drop* method does extra work to ensure that the only time a leading zero exists is when the list is entirely empty. These issues are both rectified when using a representation that uses the digits 1 and 2 rather than 0 and 1. Zero has no digits, and nonzero numbers contain only the digits 1 and 2. Again, each position is valued at twice the previous, so the initial node may hold one element or two elements, the second may hold one pair or two pairs, the third may hold one pair-of-pairs or two pairs-of-pairs, and so on. Figure 5.2 depicts a *zeroless random-access list* with eight nodes.

The pair class is unchanged from the standard random-access list. The nodes, on the other hand, differ a fair amount. Every node holds at least one piece of data, some nodes hold two. Again, the rest of the list is a list of paired values.

```

62 <zeroless random-access list internal node 62>≡ (63b)
    private static class Node<T> {
        private final boolean hasSecond;
        private final T first;
        private final T second;
        private final Node<Pair<T,T> > rest;
        <zeroless random-access list constructors 63a>
        <zeroless random-access list methods 64a>
    }

```

To accommodate the new sizes, there are two constructors: one which takes a single data value, and one which takes two. Each additionally takes a node that begins the rest of the list, which might be `null`.

<zeroless random-access list constructors 63a>≡

(62) 63a

```
public Node(T x, Node<Pair<T,T> > list) {
    hasSecond = false;
    first = x;
    second = null;
    rest = list;
}
public Node(T x, T y, Node<Pair<T,T> > list) {
    hasSecond = true;
    first = x;
    second = y;
    rest = list;
}
```

The list class itself is as follows; Node instance methods are detailed subsequently.

<lists/ZerolessRandomAccessList.java 63b>≡

63b

```
public class ZerolessRandomAccessList<E> {
    <zeroless random-access list internal node 62>
    <random-access list pair 59b>
    private Node<E> head;
    public ZerolessRandomAccessList() { head = null; }

    public int size() {
        if (head == null) { return 0; }
        return head.size();
    }
    public void prepend(E x) {
        if (head == null) {
            head = new Node<>(x, null);
            return;
        }
        head = head.prepend(x);
    }
    public E drop() {
        if (head == null) {
            throw new java.util.NoSuchElementException();
        }
        Pair<Node<E>, E> result = head.drop();
        head = result.first;
        return result.second;
    }
    public E get() { return get(0); }
    public E get(int index) {
        if (head == null) { throw new IndexOutOfBoundsException(); }
        return head.get(index);
    }
}
```

To compute the size of the list, again we must read the spine as a binary number, although in this case the digits are 1 and 2, rather than 0 and 1.

```
64a <zeroless random-access list methods 64a>≡ (62) 64b>
private int size() {
    if (rest == null) {
        if (hasSecond) { return 2; }
        return 1;
    }
    if (hasSecond) { return 2 + 2*rest.size(); }
    return 1 + 2*rest.size();
}
```

To prepend an element is again an increment operation. If the digit is a 1, it becomes a 2 and holds both the new data and the original data. If, on the other hand, the digit is a 2, it becomes a 1 holding the new data, and prepends both of its original data points together as a pair to the rest of the list.

```
64b <zeroless random-access list methods 64a>+≡ (62) <64a 64c>
private Node<T> prepend(T x) {
    if (!hasSecond) { return new Node<>(x, first, rest); }
    Pair<T,T> carry = new Pair<>(first, second);
    if (rest == null) { return new Node<>(x, new Node<>(carry, null)); }
    return new Node<>(x, rest.prepend(carry));
}
```

To drop an element is again a decrement operation. If the digit is a 2, it becomes a 1 holding the second value. The first is returned as the value to be removed. If the digit is a 1, its value is what should be returned, but, it cannot become a zero! Instead, it becomes a 2, borrowing a pair from the rest of the list. In either case, because we are dealing with persistent nodes, the new node must also be returned.

```
64c <zeroless random-access list methods 64a>+≡ (62) <64b 65>
private Pair<Node<T>, T> drop() {
    if (hasSecond) {
        T value = first;
        Node<T> node = new Node<>(second, rest);
        return new Pair<>(node, value);
    }
    if (rest == null) { return new Pair<>(null, first); }
    Pair<Node<Pair<T,T>, Pair<T,T>> >, Pair<T,T> > p = rest.drop();
    T x = p.second.first;
    T y = p.second.second;
    return new Pair<>(new Node<>(x, y, p.first), first);
}
```

Finally, the get method must account for the new node sizes. A node is never devoid of data, so if the index is zero, there will always be an immediate answer.

Table 5.2: Operations on linked lists of various sorts.

	LinkedList	RandomAccessList	ZerolessRandomAccessList
<code>size():int</code>	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
<code>prepend()</code>	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
<code>drop()</code>	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
<code>get():E</code>	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$
<code>get(int):E</code>	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$

If it is one and there are two values in the node, then again there is an immediate answer. Otherwise, the remaining index is decremented by the number of elements passed, and we pull a pair from the rest of the list. If the remaining index is even, then we want the first element, while if it is odd then we want the second.

```

<zeroless random-access list methods 64a>+≡ (62) <64c 65
private T get(int index) {
    if (index == 0) { return first; }
    if (index == 1 && hasSecond) { return second; }
    int remaining = index - 1;
    if (hasSecond) { remaining = index - 2; }
    if (rest == null) { throw new IndexOutOfBoundsException(); }
    Pair<T,T> p = rest.get(remaining / 2);
    if (remaining%2 == 0) { return p.first; }
    return p.second;
}

```

These modifications do not change the analysis of the runtime for any methods other than `get`. However, this new structure allows for constant-time `get` of the initial element, and guarantees that an arbitrary `get` is bounded not only by the log of the size of the list, but also by the log of the size of the index.

5.5 Summary

A recursive numerical function can be transformed directly into a proof of its correctness using the principle of mathematical induction, be that ordinary induction or strong induction. A recursive function in general can be transformed into a recursive numerical function describing how long it takes for that function to execute. If the structure is manipulated during processing, it might be more difficult to verify that the function is correct; using persistent data structures avoids this problem.

We examined three kinds of linked list in this chapter, parameterized by a type variable `E`. Each structure has the same five public methods: `size():int`, `prepend(E)`, `drop():E`, `get():E`, and `get(int):E`. The runtimes of these

methods are summarized in Table 5.2. The zeroless random-access list is a clear winner over the plain random-access list, being asymptotically better in one case and asymptotically equivalent in all others. However, there is no clear winner between the zeroless random-access list and the ordinary linked list. When `prepend` and `drop` operations are dominant, the ordinary list wins, but when indexing to deep values is dominant, the zeroless random-access list wins.

When operating on random-access lists of either type, we had to be careful to avoid accessing fields that are present but not valid: the `data` of the node of a plain random-access list is only valid if that node `hasData`, and the `second` of the node of a zeroless random-access list is only valid if that node `hasSecond`. In the next chapter, we will discuss how to create type families such that we can distinguish a node of one kind from a node of another kind, while declaring a common interface between them, so that the list itself need not care about which kind of node it begins with. This idea of a common interface will also be used to describe situations like we have created with our trio of lists here, where many concrete data structures solve the same problems in different ways. Each is a drop-in replacement for the others, just with different performance characteristics.